

HIPPO: A Hierarchy-Preserving and Noise-Tolerant Pre-HLS Power Modeling Framework for FPGA

Zefan Lin¹, Zedong Peng², Mingzhe Gao², Jieru Zhao^{2*}, Zhe Lin^{1*}

¹Sun Yat-sen University, Shenzhen Campus, China ²Shanghai Jiao Tong University, Shanghai, China
linzf27@mail2.sysu.edu.cn, {zedongpeng, a823337391, zhao-jieru}@sjtu.edu.cn, linzh235@mail.sysu.edu.cn

Abstract—Power estimation for customized accelerators, especially those derived from high-level programming languages, entails the invocation of a long electronic design automation (EDA) tool chain, thus incurring large timing overhead that hinders early design optimization. To mitigate this problem, in this paper, we propose HIPPO, an architecture-level power modeling framework for field-programmable gate arrays (FPGAs). HIPPO operates directly on C/C++ programs, whose execution is prior to and independent of any EDA tool including the very front-end, high-level synthesis (HLS). During power modeling, HIPPO exploits the intrinsic C/C++ code hierarchies, including nested loops and operations, and enables multi-level power estimation that aligns with different code hierarchies. Specifically, HIPPO can be decomposed into (1) a code transformation flow that directly converts a C/C++ program with HLS pragmas into hardware-oriented and power-aware control and dataflow graph, (2) a hierarchy-preserving power modeling methodology that combines analytical modeling and data-driven learning approaches to effectively orchestrate different code hierarchies, and (3) an adaptive dataflow coarsening strategy which ensures modeling accuracy, efficiency and robustness by suppressing noise of onboard measurement. Experimental results demonstrate that HIPPO effectively decomposes and accurately predicts both dynamic and total power consumption, achieving average errors of 8.89% (dynamic) and 6.31% (total) for nested loops, and 9.86% (dynamic) and 3.41% (total) for single loops, respectively. These results prove that HIPPO paves the way for power-efficient high-level architecture exploration.

Index Terms—FPGA, architectural exploration, power modeling, code hierarchy, noise management

I. INTRODUCTION

As chip density continues to increase in recent decades, the power efficiency of integrated circuits has become a pivotal design goal that severely influences their performance and also determines their longevity. This issue makes power optimization play an important role in the entire electronic design automation (EDA) process. Throughout the complete EDA tool chain, power optimization at the very early architectural design stage has been proven to be the most efficient and effective. However, in order to support early-stage power optimization, accurate power assessment is generally demanded. Unfortunately, the off-the-shelf EDA tool flow cannot well satisfy this condition. That is, the accurate power evaluation is usually accompanied by high timing costs.

Take the mainstream field-programmable gate array (FPGA) design flow [1] as an example. In an effort to evaluate the power consumption of a high-level architectural design with

C/C++ description, it is required to go through a tedious EDA tool chain, from high-level synthesis (HLS) [2], logic synthesis, placement, routing, to gate-level simulation with realistic workloads, after which power consumption is characterized in a per-gate basis. This process makes even a single round of power evaluation prohibitively time-consuming, impeding fast power optimization in the design loop that typically involves multiple rounds of architectural fine-tuning and power collection.

Existing methods. To mitigate this problem, some research works have been proposed to expedite power estimation of customized accelerator designs with high-level synthesis (HLS) methodology. Existing research works can be broadly categorized into library-based methods and learning-based methods. Library-based methods [3]–[6] attempt to construct power look-up tables (LUTs) for basic hardware components in an offline fashion. Then, when predicting the power of a holistic design, these methods look up the power consumption of individual components that constitute the complete design, and finally add up the component-level power to represent the design-level power. To build up the power LUTs, these methods involve post-implementation power analysis of each single component in particular operating conditions and thus have limited generalization ability.

In contrast, modern learning-based methods [7]–[9] seek to deduce the design-level power consumption without delving into details of micro-architectures. These works seek to extract the signal toggling of the design represented by the control and data flow graph (CDFG), a higher abstraction level of the hardware architecture compared with gate-level circuit, and apply machine learning strategies to correlate the signal toggling patterns with power consumption, aided by a variety of samples. Although these prior works demonstrate effectiveness in power estimation, their construction flow still heavily relies on the results derived from the HLS execution. Taking into account the vast number of applicable pragmas provided by the HLS tools, there is still a large timing overhead in continuously activating the HLS tool flow to traverse a number of design instances in search of architecture-level power optimization opportunities.

Main challenges. Despite the advancements in HLS-based power modeling, few of the existing works can provide portable and systematic approaches to support pre-HLS power estimation, i.e., predicting power directly using C/C++ programs without invoking any EDA tool, including the very

*Corresponding author.

front-end, HLS. Nevertheless, ensuring accurate power prediction at this stage is non-trivial as it poses two key challenges. Note that the HLS design flow encapsulates a series of hardware-oriented transformations, such as allocation, scheduling, binding and pragma-based optimization. This provides a rich set of crucial information about the register-transfer level (RTL) structure, which has a strong correlation with power consumption [10] and is widely used in prior works [7]–[9]. *Hence, the first challenge arises: without using HLS, most of the architectural details are absent, making power modeling intractable.* Moreover, by analyzing the data from power measurement, we identify non-negligible data noise characterized by random fluctuations. *This leads to the second challenge: the presence of measurement noise makes power estimation prone to error in realistic scenarios.*

Our method. To unleash the full potential of early-stage architectural exploration starting from C/C++ programs, we introduce HIPPO, a pre-HLS power modeling framework for FPGA. The rationale of HIPPO stems from a key observation that HLS’s hardware construction mechanism is naturally consistent with C/C++ code hierarchies [11]. Specifically, given a two-level nested loop with the inner- and outer-loop levels, HLS first decomposes the inner and outer loop structures into individual basic blocks, after which HLS follows a per-block basis to perform scheduling and optimization, etc. We leverage this HLS feature and further propose to explicitly maintain the C/C++ code hierarchies in power modeling, as a way of injecting domain knowledge and compensating the loss of hardware details without using HLS. Besides this, we notice that the noise in power measurement may adversely influence our hierarchical modeling flow. In light of this, an adaptive dataflow coarsening approach is investigated to counteract the negative effects of noise in data source.

To the best of our knowledge, HIPPO is the first work targeting power prediction for onboard applications based entirely on high-level C/C++ programs, which delivers high accuracy, efficiency and robustness by subtly taking into account code hierarchies while suppressing the noise of measurement. Specifically, HIPPO achieves average prediction errors of 8.89% and 9.86% for dynamic power, and 6.31% and 3.41% for total power, in nested and single loops, respectively. In short, the contributions of this paper are listed as follows:

- We develop a fully automated design flow to convert naive C/C++ programs into customized control and data flow graphs, encapsulating architecture and power information for effective modeling.
- We propose a hierarchy-preserving power modeling methodology for FPGA, which leverages analytical method and machine learning to jointly enhance the model’s representation capability on different code hierarchies.
- We introduce an adaptive dataflow coarsening approach to improve the efficiency and robustness of both total and dynamic power estimation while maintaining high noise tolerance.

II. RELATED WORK

Acquiring post-route quality of results (QoRs) and annotating them back to the architecture design level is useful in guiding the FPGA optimization process. Among the QoRs for hardware design, three metrics are of the most importance: performance, power and area (PPA). Recent studies [12]–[14] have proposed effective methods to predict the performance and resource utilization of hardware designs in both pre-HLS and post-HLS stages. Nevertheless, the estimation of power consumption is more challenging than performance and area, as it is determined by both the static architecture and the dynamic workload. The relevant research works on early power estimation can be classified into library-based methods and learning-based methods.

Library-based methods. The general idea of library-based power modeling methods is to build up a power library for each individual component, and then perform power aggregation to obtain application-level power estimation. Aladdin [4] is an early attempt to perform library-based power characterization and obtain power estimation for an entire application by adding up the inferred power of each individual hardware unit, including logic, arithmetic and memory. Similarly, Zuo *et al.* [5] extract basic tile structures in affine functions and perform tile-level power characterization. FlexCL [6] limits its scope to the OpenCL programming model in order to produce dedicated power characterization. These methods usually require careful fine-grained power characterization, with benchmarks handcrafted to fully exercise and capture the power behavior of each single component under a variety of working conditions. As a result, maintenance costs to account for changes in component specifications tend to be high.

Learning-based methods. Recent works [7]–[9] embrace data-driven approaches for HLS-based power modeling, aiming at providing power estimation for the holistic design at once rather than examining each hardware unit. The common practice of these works is to extract RTL architectural information after running HLS tool flow, identify power features from HLS intermediate results and create regression models to learn from key features that contribute to the changes in power consumption. Lee *et al.* [7] build up cycle-accurate power estimation models by specifically modeling the total power of each cycle determined in the finite-state machines. HLPow [8] captures overall architectural information from HLS intermediate results, represents signal activities as histograms, and uses decision trees and convolutional neural networks for total power inference. PowerGear [9] seeks to learn accurate dynamic power consumption by modeling interconnects via graph neural networks (GNNs). However, most of the recent research efforts have focused on post-HLS application scenarios, i.e., exploiting the rich set of architectural details generated by HLS process so as to build up power models.

Summary. The comparison between our approach and representative works is detailed in Table I. Only the works [4] and [5] are applicable in the pre-HLS stage but both of them suffer from high labor costs in modeling and maintenance,

TABLE I: Comparison between Our Work and Representative Works.

Method	Target	Input Stage	Modeling Granularity	Prediction Objective	Noise Tolerance
ISCA'14 [4]	ASIC	Pre-HLS	Fine	Total Power	Low
ICCAD'15 [5]	ASIC	Pre-HLS	Coarse	Total Power	Low
DATE'15 [7]	ASIC	Post-HLS	Fine	Total Power	Low
TCAD'23 [8]	FPGA	Post-HLS	Coarse	Total Power	Low
DATE'22 [9]	FPGA	Post-HLS	Coarse	Total & Dynamic	Low
Ours	FPGA	Pre-HLS	Hierarchical	Total & Dynamic	High

and also indicate a lack of transferability. As for post-HLS modeling [7]–[9], although they speed up the prediction, the timing cost of continuously running the HLS tool flow is still significant when it comes to design space exploration for large-scale designs with a rich set of optimization pragmas. This paper aims to bridge this gap by proposing a comprehensive pre-HLS power estimation method that is able to infer both total and dynamic power in a hierarchical manner. Moreover, this paper presents the first attempt to study noise-tolerant power modeling strategy applicable to real-world scenarios.

III. METHODOLOGY

The overview of our pre-HLS power modeling approach, HIPPO, is depicted in Fig. 1. The complete flow can be roughly decomposed into model feature construction and power label extraction according to the functionality of different components.

The model feature construction flow (upper half of Fig. 1) takes C/C++ application programs, the related testbenches and the pragma configurations as the input to generate a customized CDFG called HP-CDFG, power-related features and coarsened loop structures, which together form a feature graph that supports both model training and inference.

The power label extraction stage (lower half of Fig. 1) is deployed for power model training only. In this stage, we collect a set of training samples from various operations and applications described by C/C++ programs. Note that by applying a diverse set of pragmas to each application, we can obtain multiple data points per application, each exhibiting unique power behavior. We then execute the complete EDA flow from HLS to post-implementation to generate the bitstreams for all design points and operations. After that, we implement the bitstreams on board and measure the real-time power consumption. This gives us the ground-truth power values for the design points and operations. Then, we can bundle the generated feature graph and the corresponding ground-truth power into a sample. Finally, we leverage all the samples to develop analytical models (for operations) and GNNs models (for loops) in a hierarchical manner.

In the prediction stage, in order to estimate the power for new designs of interest, we only need to use the model feature construction flow to generate the feature graphs, after which we can directly utilize the well-trained models to conduct power prediction for different loop levels.

To ensure accurate, robust and efficient power modeling, we investigate three key techniques in this paper: (1) a *customized CDFG construction strategy*; (2) a *hierarchy-preserving power*

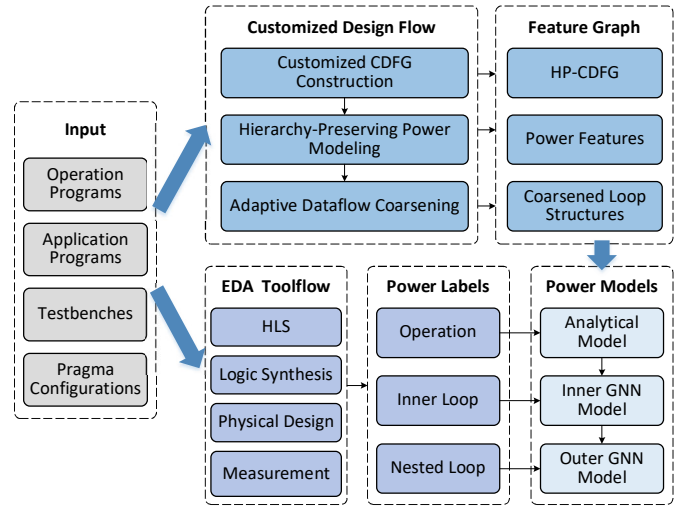


Fig. 1: The overall design flow of HIPPO.

modeling methodology; and (3) an *adaptive dataflow coarsening approach*.

A. Customized CDFG Construction

The key to achieving accurate power estimation at the architectural level is to embed as much hardware and power detail as possible, which is challenging since the C/C++ programs themselves do not contain such information. To circumvent this issue, we apply hardware domain knowledge to help establish a hardware-oriented and power-aware graph structure from scratch, which we refer to as *HP-CDFG* in our paper. The construction of HP-CDFG can be decomposed into the following three steps.

Generate and simplify CDFG. Starting with a C/C++ program with HLS pragmas specified, the first step of our flow is to construct its CDFG. A CDFG contains a control flow that describes the transitions between basic blocks and a dataflow that depicts the data dependency between arithmetic, logic and memory operations, which serves as an intermediate layer between hardware and software. We use the Clang front-end to transform the source code into LLVM intermediate representation (IR) [15]. Then, the LLVM IR is further processed by ProGraML [16] to produce an initial CDFG.

In addition, we notice that this naive CDFG contains a lot of auxiliary nodes and edges that complicate the graph structure but only bring marginal benefit to our power modeling target. Therefore, we perform a two-level CDFG simplification. At the inter-block level, we add the entry (*call*) and exit (*br/ret*) nodes to represent each basic block explicitly. Subsequently, we replace the edge flow between nodes in different blocks with block-level flow that passes through entry nodes and exit nodes. Therefore, the inter-block transitions are outlined more clearly. We also substitute the inter-block control flow by edges between the entry and exit nodes to simplify the inter-block control flow. At the intra-block level, we remove the control flow representing the adjacency between IR instructions, which is useful for program understanding but not for hardware analysis. Fig. 2 shows how we process the input code, construct the CDFG and simplify it for our task.

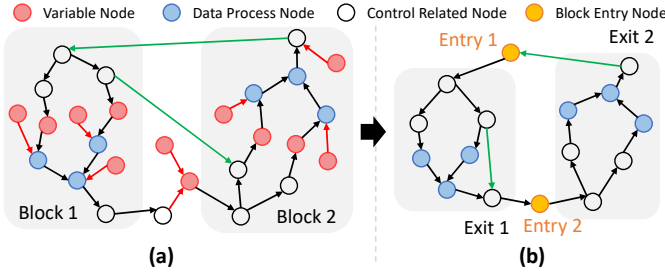


Fig. 2: CDFG construction and simplification.

Embed HLS pragmas to reflect structural changes.

HLS pragmas can cause structural changes in the generated hardware. We account for this effect in the construction of HP-CDFG. To this end, we identify two HLS pragmas that significantly contribute to the formation of architectural differences, i.e., *loop unrolling* and *array partitioning*. The former alters a loop structure by replicating the loop body multiple times, which reflects the duplication of arithmetic and logic instances in hardware, while the latter splits a holistic array into multiple independent memory elements that operate in parallel, which corresponds to the increase in memory utilization.

To account for loop unrolling, we extract the corresponding loop unrolling pragmas applied to C/C++ programs in HLS, and profile the operated loop scope and unrolling factor for each of these pragmas. Then, we follow the same user-specified strategy and instrument the original C/C++ programs with LLVM-compatible loop unrolling instructions. Moreover, based on the fact that an inner loop is completely unrolled once an outer loop is pipelined in HLS, we additionally identify this pragma combination through profiling the code and HLS pragmas, and instantiate the LLVM instructions to fully unroll the inner loop in this situation. In this way, the loop unrolling behaviors can be properly integrated in the IR upon LLVM compilation with the C/C++ programs.

To deal with array partitioning, we notice that the initial CDFG constructed by ProGraML does not contain any detail about the arrays, which is not hardware-friendly. To properly infer memory behaviors, we profile the use of arrays in the source program and manually insert nodes in CDFG to represent arrays. The array partitioning mechanism is also taken into account by instantiating a new node for each partitioned array element. For a k -dimensional array, if the array partitioning pragmas specify $p_1, p_2, \dots, p_k$ as the partition factors for the k dimensions in the given order, we correspondingly add $\prod_{i=1}^{n=k} p_i$ nodes to the graph to represent the array's memory ports. Furthermore, we establish edges between the newly inserted memory nodes with the existing load and store nodes in the initial CDFG to reflect the actual dataflow from memories to operations. Specifically, we mark each array element with its partitioned dimension and assign an index to it. Then, in the LLVM IR, we retrieve the memory addresses of load and store operations, through which we recover the dimension and index of the accesses. This helps us to pair up array elements and memory accesses. Finally,

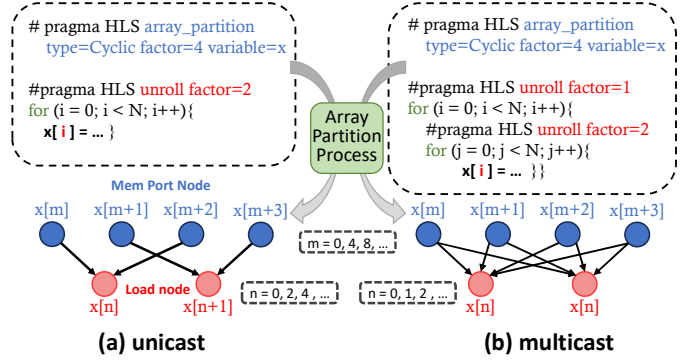


Fig. 3: Memory accesses patterns: unicast and multicast.

edges from the array elements to their corresponding memory access operations are added to the CDFG. We summarize the memory accesses into two patterns, i.e., *unicast* and *multicast*, as shown in Fig. 3.

Integrate signal switching for power indication. At present, the constructed CDFG is static only and cannot reflect power changes under real workloads. From a hardware perspective, the dynamic power consumption, which constitutes a significant proportion of the total power, is closely associated with signal switching within and between hardware components. To indicate the circuit status changes and thereby power consumption under realistic workloads, we apply the *Hamming distance* [17] to quantify consecutive signal switching. The Hamming distance is formally defined as:

$$HD(t, t-1) = \sum_{i=1}^N \delta(S_t, S_{t-1}), \quad (1)$$

where $HD(\cdot)$ represents the Hamming distance, S denotes the signal of interest, N means the signal bitwidth, and t is the time stamp. In the context of signal switching, the Hamming distance reflects the variations in all the bits of a signal between two consecutive time stamps, from $t-1$ to t . Therefore, taking the average of the Hamming distance over a specified time period is a viable solution to quantifying the signal switching within that period, which we define as *switching activity* (SA) and it is formulated as:

$$SA_s = \frac{\sum_{t=1}^T HD(t, t-1)}{T}, \quad (2)$$

where suffix s denotes a signal of interest and T represents the total execution cycles (overall latency) of the target application.

We build up a compilation-based switching activity extraction flow and then infer the cycle-level changes of all the signals of interest in the dataflow of CDFG, which can be viewed as a fast and simplified substitute of RTL simulation. Specifically, we instrument the LLVM IR with a *tracer* function for each arithmetic, logic and memory operation in IR, in order to keep track of the real values for each operand that produces a signal to trace. The instrumented IR, in conjunction with a testbench that incorporates input stimuli, is compiled into a binary. We then execute the binary, and whenever a

traced operation is activated in the program, the values of its operands are recorded by the tracer, from which we can retrieve the Hamming distance per operand according to Eq. 1. After a certain time period of execution, we can extract the switching activity of a signal according to Eq. 2. Note that our flow is similar to that of PowerGear [9] in signal tracing, but PowerGear operates on the HLS intermediate results and infers the signal switching according to hardware instances, whereas our flow targets an earlier and more general IR stage, independent of the hardware platform and EDA flow.

B. Hierarchy-Preserving Power Modeling

Power consumption is known to be highly sensitive to the micro-architectures and interconnects within the hardware design [10]. This inherent characteristic poses a significant challenge in achieving high accuracy in architecture-level power estimation, particularly in the pre-HLS stage where all hardware-related information has been abstracted. To address this challenge, we propose to incorporate domain knowledge into power modeling to bridge the gap between software programs and hardware implementation.

Our core idea stems from the fact that the hardware generation mechanism of HLS is in accordance with the hierarchies of the input C/C++ code [11]. In the front-end of HLS, given a nested loop structure, the HLS tools first decompose the holistic loop into a series of basic blocks, where the inner loop segment without branches is transformed into an individual basic block in the IR. Similarly, the outer loop body that contains operations in between two loop levels is converted into a basic block. In the back-end of HLS, given that the major tasks, such as scheduling and binding, are NP-Complete or NP-Hard problems, the HLS tools employ a divide-and-conquer paradigm that first generates an optimal scheduling and binding result for each basic block, and then combines these block-level intermediate results to form the overall solutions for scheduling and binding. This has prompted us to enhance the architecture-level power modeling capabilities by maintaining the code hierarchies.

Overall, we decompose a C/C++ program into three levels: *operation level*, *inner loop level* and *nested loop level*. Each level is indicative of a specific degree of granularity, ranging from the finest to the coarsest. Most importantly, the predicted power of a lower level element will be exploited in the higher level structure as useful power features. We apply different methods to estimate the power consumption for different levels: for the operation level, we develop analytical models for quick estimation, whereas for the other two levels, as the problem becomes more intricate, we use advanced GNNs to learn power behaviors from handcrafted features. This hierarchy-preserving power modeling method is shown in Fig. 4.

Analytical power model at the operation level. The arithmetic and memory operations constitute the fundamental components of the data paths, especially for computation-intensive applications. In addition, these operations have relatively simple micro-architectures in contrast to the loop struc-

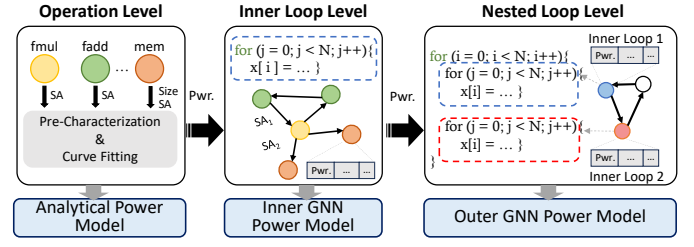


Fig. 4: Hierarchy-Preserving Power Modeling.

tures, so we develop analytical models to describe operation-level power consumption.

For arithmetic operations, as they have relatively fixed micro-architectures on FPGA, we develop a linear regression model for each type of arithmetic operation (*fadd*, *fmul*, etc.), to associate their power consumption with the operations' switching activity as follows:

$$P(op) = C_{dyn} \cdot \frac{\sum_i^N SA_{op_i}}{N} + C_{stat}, \quad (3)$$

where N is the number of operands of the operation, C_{dyn} and C_{stat} are two coefficients representing dynamic power and static power, respectively. $\frac{\sum_i^N SA_{op_i}}{N}$ computes the average switching activities of all the operands of the evaluated arithmetic operator op .

For memory operations including *load* and *store*, given that each memory has a distinct memory size, the power consumption is determined by both the memory size and the switching activities from memory accesses. Herein, we estimate the power of the whole memory node that we insert in the HP-CDFG instead of a single load or store operation. The memory power model is developed as follows:

$$P(mem) = C_{dyn} \cdot \frac{\sum_i^N SA_{mem_i}}{N} + C_{size} \cdot S_{mem} + C_{stat}, \quad (4)$$

where S_{mem} is the memory size, N is the number of memory operations including load and store, C_{dyn} , C_{size} and C_{stat} are three coefficients representing dynamic power, size-related static power and the constant static power, respectively. $\frac{\sum_i^N SA_{mem_i}}{N}$ computes the average switching activities of all the load and store operations that access the evaluated memory mem . Noting that the memory can be implemented in two ways, i.e., block random-access memory (BRAM) or LUT, we need to model each type separately.

In order to acquire the operation-level power to determine the coefficients in the aforementioned arithmetic and memory models, we simultaneously implement more than one hundred operations in hardware and we take their average power as the ground-truth power. We exercise the same type of operation with different switching activities to calibrate the power models. Finally, all of our operation-level power models demonstrate a R^2 value of more than 0.92, indicating a high correlation between estimation and real power.

GNN power model at the inner and nested loop levels. Loops can contain multiple operations with intricate data dependencies, so their complexity is orders of magnitude

higher than that of a single operation or memory element. In addition, there are various types of interconnection structures in the loop, which are difficult to analyze comprehensively in advance. These two reasons make it difficult or even impossible to build an analytical power model for loop structures. Noting that HP-CDFGs are inherently graph-structured data, we can readily transform them into training samples and employ GNNs to learn the diverse power behaviors of loops.

We first build up the GNN power model at the inner loop level. There are three cases that can be mapped to an inner loop: (1) a single loop, (2) the innermost loop in a nested loop, and (3) a nested loop with the pragmas of loop pipelining or flattening that removes the inner loop hierarchies. Based on the HP-CDFGs constructed earlier, we further embed node features and edge features into each HP-CDFG and develop GNNs that perform power learning and inference on the feature-annotated HP-CDFGs. The node features are *latency*, *delay*, *LUT*, *DSP*, *FF*, *total power* and *dynamic power*. The edge features are *switching activities* of the signals associated with the edges, which is computed according to Eq. 2.

Regarding the nested loop level, we leverage the predicted power of the inner loop level as a feature to deduce the power for the nested loop structure. This is based on the heuristic that a nested loop invokes its inner loop body in a coarser granularity, treating the inner loop as a single but compound operation. During the model development, we also find that the noise in the inner loop level may adversely affect the power estimation accuracy, and this will further harm the prediction accuracy of the nested loop as we utilize the power estimation of the inner loop as a feature. To alleviate this situation, we additionally integrate an adaptive dataflow coarsening flow, as shown in Section III-C.

C. Adaptive Dataflow Coarsening Approach

In practical scenarios, the presence of measurement noise can inevitably compromise the accuracy of power modeling. In our proposed hierarchical modeling strategy, the estimation results of inner loops are leveraged as features for the prediction of the entire nested loop. Consequently, the estimation error may be amplified when the nested loop utilizes inaccurate features from the inner loop. To reduce the effect of noise while still making use of the hierarchical features of the applications, an adaptive and noise-tolerant dataflow coarsening approach is proposed as an augmentation to the hierarchical modeling method. We first quantify the effect of noise in power measurement, and then explain our solution.

Effect of noise in power measurement. We first analyze the noise in power measurement. Fig. 5 shows the measured dynamic power over time for two representative design points, with average power consumption of 10 mW (design-a in Fig. 5 (a)) and 113 mW (design-b in Fig. 5 (b)), respectively. The dynamic power of design-a fluctuates between 4 mW and 14 mW, with a variance of 4.5 mW. Similarly, the dynamic power of design-b ranges between 108 mW and 118 mW, and its variance is 6.4 mW. In both cases, the maximum observed discrepancy in measured power is 10 mW. However,

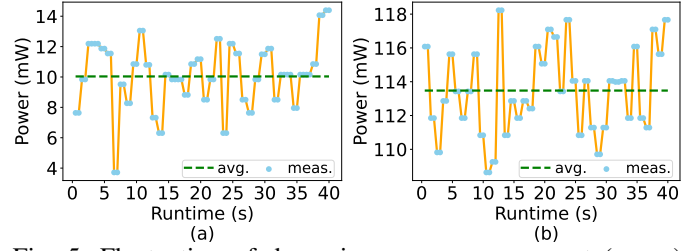


Fig. 5: Fluctuation of dynamic power measurement (meas.) over time: (a) design with an average (avg.) power of 10 mW; (b) design with an average power of 113 mW.

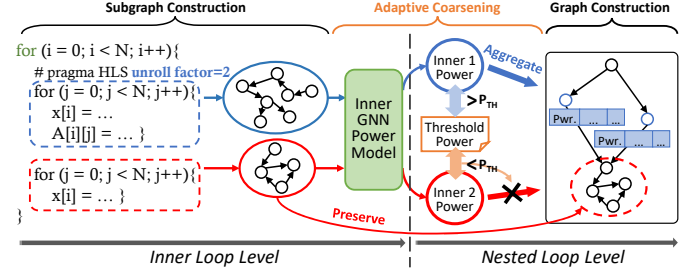


Fig. 6: Adaptive dataflow coarsening strategy in HIPPO.

the impact of noise varies between these two designs: design-a demonstrates a substantial response to noise, while design-b exhibits a less pronounced effect. This observation gives rise to our proposal of the adaptive dataflow coarsening strategy.

Adaptive dataflow coarsening. The adaptive dataflow coarsening strategy is employed in combination with the hierarchical modeling to suppress noise and thereby enhance the accuracy of power prediction for the complicated nested loops. This strategy comprises two main components: fine-grained power assessment and dataflow coarsening.

For a nested loop, the initial step entails the decomposition of the loop into its inner and outer hierarchies. Given that we have already built up the analytical models at the operation level and the GNN predictive model for the inner loop level, as discussed in Section III-B, we can now obtain the inner-loop power estimation from the pre-trained models. Next, we compare the prediction result against a threshold to evaluate the effect of noise, which is set to 10 mW in this paper. If the prediction result is below the threshold, we consider the prediction for the inner loop significantly influenced by noise. Therefore, the estimation result is not a good reference for the outer layer.

Subsequently, the inner loop undergoes a selective coarsening stage to form a *super node* in the nested loop. The decision of whether to establish a super node is contingent upon the estimated power for the inner loop hierarchy: If the estimated power of the inner loop is larger than the aforementioned noise-relevant threshold, the loop is condensed into a super node with latency, total power, dynamic power used as the node features, and vice versa. The super node inherits the original data dependencies with other parts of the application, and if the outer loop is unrolled, the super node is replicated to comply with this behavior, as shown in Fig. 6. Finally, we train a unified GNN to infer the power consumption of the nested loops, as shown on the rightmost side of Fig. 4.

TABLE II: Accuracy of the power estimation at the nested-loop level with HIPPO using different GNNs.

Dataset	Test Size	PNA				GINE				GraphSage			
		Dynamic		Total		Dynamic		Total		Dynamic		Total	
		NMAE	NRMSE	NMAE	NRMSE	NMAE	NRMSE	NMAE	NRMSE	NMAE	NRMSE	NMAE	NRMSE
atax	105	11.12%	0.22	5.28%	0.10	19.38%	0.45	11.09%	0.26	35.95%	0.81	6.30%	0.11
bicg	116	13.65%	0.33	4.11%	0.08	21.78%	0.38	12.84%	0.23	49.39%	1.18	6.38%	0.11
gemm	92	23.82%	0.86	17.53%	0.86	19.36%	0.55	10.34%	0.29	36.25%	1.32	6.40%	0.24
gesummv	58	15.99%	0.23	4.71%	0.07	38.74%	0.64	6.27%	0.08	30.67%	0.45	6.53%	0.09
k2mm	75	17.68%	0.43	2.57%	0.04	21.14%	0.38	8.18%	0.19	38.90%	1.17	5.71%	0.16
k3mm	80	24.47%	0.92	6.02%	0.14	30.58%	0.90	20.51%	0.47	29.77%	0.69	11.11%	0.30
mvt	114	17.74%	0.65	3.16%	0.06	29.52%	1.33	8.08%	0.23	25.69%	0.67	4.73%	0.12
syr2k	51	14.98%	0.46	5.68%	0.17	46.30%	1.96	11.02%	0.23	29.07%	0.67	5.15%	0.09
syrk	55	4.33%	0.10	12.43%	0.51	24.76%	0.71	6.70%	0.14	47.58%	1.97	19.37%	0.78
merge	225	7.22%	0.11	5.58%	0.09	15.26%	0.22	11.64%	0.15	17.60%	0.27	10.68%	0.16
overall	971	8.89%	0.26	6.31%	0.25	16.98%	0.43	11.35%	0.23	21.07%	0.61	9.34%	0.23

In summary, the benefits of our adaptive dataflow coarsening strategy are three-fold. Firstly, we subtly make use of the reliable power estimation from the lower loop levels to help calibrate the power consumption of the more complicated outer loop level. This practice ensures high power modeling accuracy by compensating the loss of hardware details in this early architectural design stage. Secondly, our strategy is more robust than a method that casts all inner loops as super nodes, as employed for latency and resource prediction in [18]. This is because measurement noise tends to be amplified from the inner loop to the outer loop, while our strategy can filter out the error-prone prediction results, maintaining high noise tolerance throughout power modeling. Thirdly, inner loop with high dynamic power usually means that more operations are engaged in this loop and these loops are the main objects for node coarsening. By selectively compressing the highly complex inner loop structure into a coarsened super node, we can simplify GNN training and also streamline power estimation, thus yielding high estimation efficiency.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

HIPPO achieves a fully automated design flow for pre-HLS power estimation with C/C++ source code as the input. The complete design flow is implemented by C++ and Python. We select a number of realistic applications from Polybench [19] and Machsuite [20] to form our evaluation datasets, which are widely used for HLS design prototyping. We add the HLS pragmas of loop pipelining, loop unrolling, array partitioning and loop flattening for each application to generate a rich set of design points. To evaluate the scalability, we also randomly merge design points to form larger designs at scale. We use AMD/Xilinx Vitis-HLS 2022.2 and Vivado 2022.2 to execute the complete EDA design flow. Finally, we generate 7,090 design points for inner loop level and 9,696 design points for the application level in different applications with different combinations of pragmas. These designs are all implemented on a real AMD/Xilinx ZCU102 board and we collect ground-truth power values through onboard measurement using the Power Advantage Tool [21].

Regarding the modeling process, the C/C++ application programs together with the employed HLS pragmas are fed into HIPPO design flow for power learning and inference.

For each GNN model, we use 80% of the design points for training, 10% for validation and 10% for testing. We examine our method with both the total power and dynamic power as the prediction objectives. We develop our GNN regression models with three graph convolution layers, followed by a pooling layer with the combination of max and sum pooling, and a multilayer perceptron (MLP) layer at last. We train each GNN model with the power-featured graphs generated by HIPPO. The training procedure experiences 250 epochs and we save the best model according to the validation results.

B. Accuracy of HIPPO at the Nest-Loop Level

We follow our HIPPO methodology to train the GNN models of both the inner-loop level and nested-loop level using the training sets. The prediction objectives include the dynamic and the total power (dynamic plus static) consumption. After the models are generated, we predict the power consumption of the test sets to evaluate the efficacy of our method. This guarantees that our training method is not biased towards the samples for testing. Specifically, we examine three widely used GNN variants: PNA [22], GIN with edge features (GINE) [23] and GraphSage [24] at the nested-loop level. Metrics to evaluate our models include the normalized mean absolute error (NMAE), the normalized root mean square error (NRMSE) and the coefficient of determination (R^2) [25]. These three metrics quantify the error percentage, the magnitude of deviations and the goodness of fit, respectively, which are widely used for evaluating power models [26]. NMAE and NRMSE are formally defined as:

$$\text{NMAE} = \frac{\sum_{i=1}^N |y_i - \hat{y}_i|}{\sum_{i=1}^N y_i}, \text{NRMSE} = \frac{1}{\bar{y}} \sqrt{\frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{N}}, \quad (5)$$

where y_i and $\hat{y}_i$ are the ground-truth and predicted value of power, respectively, and N is the size of sample.

Table II demonstrates the results of power estimation. HIPPO with the PNA model achieves the best prediction accuracy for the dynamic power estimation, with the average NMAE of 8.89%, the average NRMSE of 0.26 and the R^2 of 0.99. Based on the fact that the PNA is superior to the GINE and GraphSage in its learning capability of edge features, the results serve to validate the effectiveness of HIPPO in accurately capturing switching activities as power indicators on edges. As for total power consumption, HIPPO with the

TABLE III: Comparison with SOTA works.

Method	Modeling Stage	Estimation Error
HL-Pow [8]	Post-HLS	4.82%
PowerGear [9]	Post-HLS	3.60%
HIPPO	Pre-HLS	4.05%

TABLE IV: Power estimation results at the inner-loop level.

Method	Target	NMAE	NRMSE	R^2
With All Features	Dynamic	9.86%	0.07	0.992
	Total	3.41%	0.06	0.994
Without Op. Features	Dynamic	17.70%	0.45	0.989
	Total	4.72%	0.09	0.987

PNA model achieves the lowest NMAE of 6.31%, whereas the GINE and GraphSage show slightly better NRMSE of 0.23. The disparities among the three models are less pronounced due to the diminished impact of edge features on total power consumption. Conversely, structural features have greater significance in static power analysis, which is also successfully encapsulated in HIPPO.

We compare HIPPO with the closest state-of-the-art (SOTA) works, HL-Pow [8] and PowerGear [9]. We conduct end-to-end comparison on the same set of benchmarks and use the same metric to compute the estimation error of the total power consumption. Experimental results in Table III demonstrate that HIPPO exhibits comparable performance to that of the SOTA works, with a maximum discrepancy of 0.45% in comparison to PowerGear. However, both of these two prior works operate on the post-HLS stages, where a rich set of architectural details can be utilized. In contrast, HIPPO targets the pre-HLS stage, which can improve the efficiency significantly with only a negligible loss in accuracy.

In general, the architecture-level power estimation has posed a significant challenge, particularly in the absence of hardware specifications at the pre-HLS stage. The experimental results of HIPPO fully justify its capability in delivering accurate, efficient and robust dynamic and total power estimation without the necessity of activating EDA tools, which opens up more opportunities for early-stage design optimization.

C. Accuracy of HIPPO at the Inner-Loop Level

In addition to predicting the overall power, as is typical in most power estimation works, HIPPO also enables power prediction for a more fine-grained loop hierarchy. In this experiment, we evaluate the efficacy of HIPPO in the power prediction at the inner loop level. To examine the effect of operation-level features, we also implement a variant of HIPPO by simply removing the features associated with each node/operation, including the operation-level power estimation by the analytical models and the switching activities associated with the operations.

Table IV demonstrates the accuracy of power prediction at the inner loop level. The results show that HIPPO achieves high prediction accuracy for both the dynamic and total power consumption at the inner loop level, with the average NMAE of 9.86% and 3.41%, respectively. The high R^2 values for both objectives further verify HIPPO's hierarchical learning capability. Conversely, in the absence of operation-level features, the errors in power estimation are observed to rise

TABLE V: Ablation study on the coarsening strategy.

Coarsening Strategy	Target	NMAE	NRMSE	R^2
Adaptive Coarsening	Dynamic	8.89%	0.26	0.999
	Total	6.31%	0.25	0.848
Full Coarsening [18]	Dynamic	13.79%	0.36	0.989
	Total	4.96%	0.13	0.947

considerably, from 9.86% to 17.70% for dynamic power and from 3.41% to 4.72% for total power. The decrease in the accuracy of dynamic power prediction is more pronounced due to its sensitivity to switching activities. Therefore, the elimination of the activities has been demonstrated to have a substantial negative impact on the accuracy of the prediction.

D. Ablation Study

HIPPO also incorporates an adaptive coarsening approach, as illustrated in Section III-C. In this experiment, we assess the efficacy of the coarsening strategy in facilitating model development. We conduct an ablation study to replace the adaptive coarsening strategy with a full coarsening method [18] that has been used for the prediction of other QoRs. Specifically, this full coarsening method always transforms the inner loop structure into a super node, with its corresponding power prediction incorporated as part of the power features.

As demonstrated in Table V, replacing the adaptive coarsening method with the full coarsening method increases the NMAE of dynamic power estimation from 8.89% to 13.79%, leading to an approximate 5% degradation in accuracy. Interestingly, an enhancement has been observed in total power estimation, with the NMAE decreasing from 6.31% to 4.96%. This experimental result is consistent with our observation that dynamic power consumption is more sensitive to noise in the real environment, which can be properly handled using our adaptive coarsening approach. Since the prediction of the dynamic power is typically a more difficult task over total power, our method demonstrates high robustness with a significant improvement in modeling dynamic power consumption.

V. CONCLUSION

In this paper, we propose HIPPO, a pre-HLS power modeling approach for FPGA. HIPPO is characterized by three distinguishing features. Firstly, HIPPO integrates a design flow to generate customized CDFGs that are effective for both dynamic and total power modeling. Secondly, HIPPO introduces a hierarchy-preserving power modeling strategy, which combines operation-level analytical models and loop-level GNN models in a hierarchical manner. Thirdly, HIPPO describes an adaptive dataflow coarsening strategy to maintain high noise tolerance in real scenarios. Experimental results fully verify that HIPPO is accurate, efficient and robust.

VI. ACKNOWLEDGMENT

This work is supported by the National Natural Science Foundation of China (No. 62404257, No. 62472273), the Shenzhen Science and Technology Program (Grant No. RCBS20231211090600003), and the Guangdong Basic and Applied Basic Research Foundation (2023A1515110769, 2024A1515013155).

REFERENCES

- [1] AMD Ltd, *Vivado design suite tutorial: Power Analysis and Optimization (UG997)*. AMD White Paper, 2024.
- [2] P. Coussy and A. Morawiec, *High-Level Synthesis: From Algorithm to Digital Circuit*. Springer, 2008.
- [3] D. Chen, J. Cong, Y. Fan, and Z. Zhang, “High-level power estimation and low-power design space exploration for fpgas,” in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2007, pp. 529–534.
- [4] Y. S. Shao, B. Reagen, G.-Y. Wei, and D. Brooks, “Aladdin: a pre-rtl, power-performance accelerator simulator enabling large design space exploration of customized architectures,” in *The 41st Annual International Symposium on Computer Architecture (ISCA)*, 2014, p. 97–108.
- [5] W. Zuo, W. Kemmerer, J. B. Lim, L.-N. Pouchet, A. Ayupov, T. Kim, K. Han, and D. Chen, “A polyhedral-based systemc modeling and generation framework for effective low-power design space exploration,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2015, pp. 357–364.
- [6] Y. Liang, S. Wang, and W. Zhang, “Flexcl: A model of performance and power for opencl workloads on fpgas,” *IEEE Transactions on Computers (TC)*, vol. 67, no. 12, pp. 1750–1764, 2018.
- [7] D. Lee, L. K. John, and A. Gerstlauer, “Dynamic power and performance back-annotation for fast and accurate functional hardware simulation,” in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2015, pp. 1126–1131.
- [8] Z. Lin, T. Liang, J. Zhao, S. Sinha, and W. Zhang, “HI-pow: Learning-assisted pre-rtl power modeling and optimization for fpga hls,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 42, no. 11, pp. 3925–3938, 2023.
- [9] Z. Lin, Z. Yuan, J. Zhao, W. Zhang, H. Wang, and Y. Tian, “Powergear: Early-stage power estimation in fpga hls via heterogeneous edge-centric gnns,” in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2022, pp. 1341–1346.
- [10] D. Liu and C. Svensson, “Power consumption estimation in cmos vlsi chips,” *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 29, no. 6, pp. 663–670, 1994.
- [11] AMD/Xilinx Ltd, “Vitis high-level synthesis user guide,” White Paper, UG1399 (v2024.2), 2024.
- [12] N. Wu, H. Yang, Y. Xie, P. Li, and C. Hao, “High-level synthesis performance prediction using gnns: benchmarking, modeling, and advancing,” in *The 59th ACM/IEEE Design Automation Conference (DAC)*, 2022, p. 49–54.
- [13] A. Sohrabzadeh, Y. Bai, Y. Sun, and J. Cong, “Robust gnn-based representation learning for hls,” in *IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [14] E. Murphy and L. Josipović, “Balor: Hls source code evaluator based on custom graphs and hierarchical gnns,” in *The 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.
- [15] C. Lattner and V. Adve, “Llvm: A compilation framework for lifelong program analysis & transformation,” in *International symposium on code generation and optimization (CGO)*, 2004, pp. 75–86.
- [16] C. Cummins, Z. V. Fisches, T. Ben-Nun, T. Hoefer, M. F. O’Boyle, and H. Leather, “Programl: A graph-based program representation for data flow analysis and compiler optimizations,” in *International Conference on Machine Learning (ICML)*, 2021, pp. 2244–2253.
- [17] A. Bookstein, V. A. Kulyukin, and T. Raita, “Generalized hamming distance,” *Information Retrieval*, vol. 5, pp. 353–375, 2002.
- [18] M. Gao, J. Zhao, Z. Lin, and M. Guo, “Hierarchical source-to-post-route qor prediction in high-level synthesis with gnns,” in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024, pp. 1–6.
- [19] L.-N. Pouchet. (2012) Polybench: The polyhedral benchmark suite. [Online]. Available: <http://www.cs.ucla.edu/%7Epouchet/software/polybench>
- [20] B. Reagen *et al.*, “MachSuite: Benchmarks for accelerator design and customized architectures,” in *IEEE International Symposium on Workload Characterization (IISWC)*, 2014, pp. 110–119.
- [21] Xilinx Ltd, “Zynq UltraScale+ MPSoC Power Advantage Tool 2018.1,” Technical Report, 2018.
- [22] G. Corso, L. Cavalleri, D. Beaini, P. Liò, and P. Veličković, “Principal neighbourhood aggregation for graph nets,” in *Advances in neural information processing systems (NeurIPS)*, 2020, pp. 13 260–13 271.
- [23] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *The International Conference on Learning Representations (ICLR)*, 2019.
- [24] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in neural information processing systems (NeurIPS)*, 2017, p. 1025–1035.
- [25] N. J. Nagelkerke *et al.*, “A note on a general definition of the coefficient of determination,” *biometrika*, vol. 78, no. 3, pp. 691–692, 1991.
- [26] Z. Xie, X. Xu, M. Walker, J. Knebel, K. Palaniswamy, N. Hebert, J. Hu, H. Yang, Y. Chen, and S. Das, “Apollo: An automated power modeling framework for runtime power introspection in high-volume commercial microprocessors,” in *The 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 1–14.